



UNIVERSIDAD ESTATAL A DISTANCIA
VICERRECTORÍA ACADÉMICA
ESCUELA DE CIENCIAS EXACTAS Y NATURALES



Cátedra Tecnología de Sistemas

Proyecto # 1. Valor 2%

Temas de Estudio

Tema 1: Fundamentos de C++

Subtemas

1. Introducción a las computadoras y a C++

- Historia y evolución de las computadoras.
- Conceptos básicos de hardware y software.
- Introducción al lenguaje de programación C++.

2. Introducción a la programación en C++, entradas/salidas y operadores

- Estructura básica de un programa en C++.
- Uso de cin y cout para entradas y salidas.
- Operadores aritméticos, relacionales y lógicos.

3. Introducción funciones miembro y cadenas

- Definición y uso de funciones.
- Manejo de cadenas de caracteres (string).

Tema 2: Algoritmos e instrucciones de control

Subtemas

1. Desarrollo de algoritmos e instrucciones de control: Parte 1

- Concepto de algoritmo.
- Operadores matemáticos y relacionales.
- Estructuras de control: if, else, switch.

2. Estructuras de control; Parte 2: operadores lógicos

- Bucles: for(), while(), do-while().
- Operadores lógicos y su uso en condiciones.

Objetivo

El propósito de este proyecto es que el estudiante desarrolle un programa en C++, poniendo en práctica los conceptos aprendidos en los temas de estudio. A través de esta experiencia, fortalecerá su comprensión y habilidades en el uso de los estudiado, aplicándolo en un contexto práctico y funcional.

Software de Desarrollo

Las instrucciones para la instalación de Code::Blocks, están disponibles en el campus virtual AprendeU.

Desarrollo

1. Descripción del Problema

Se requiere desarrollar un sistema en C++ que funcione como un simulador de cajero automático. Este debe emular las funciones esenciales de un cajero automático, tales como:

- Validación de PIN
- Consulta de saldo
- Retiros de dinero
- Depósitos de dinero.

2. Requerimientos funcionales

2.1. El sistema debe mostrar un mensaje de bienvenida e indicar al usuario que ingrese un Pin de 4 dígitos como medida de seguridad. Se asume que el usuario ha insertado su tarjeta previamente en la ranura del cajero.

- Se deben crear máximo 5 PINs válidos correspondientes a 5 clientes ficticios.
- Estos PINs puede definirse mediante constantes, opcionalmente, el estudiante puede usar un vector para almacenarlos, pero no es requisito obligatorio.
- El usuario tendrá un máximo de 3 intentos para ingresar un PIN correcto.
- Si excede este límite, el sistema debe mostrar un mensaje y cerrar el programa automáticamente.
- Opcional: Se puede implementar que el PIN ingresado no se visualice. Ejemplo: ****

```

=====
BIENVENIDO AL SIMULADOR DE CAJERO AUTOMÁTICO
=====
Por favor, ingrese su PIN (4 dígitos): _

```

2.2. Una vez validado el PIN, el sistema debe mostrar un menú principal con 5 opciones, a través del cual el usuario podrá realizar diferentes operaciones. Este menú se repetirá hasta que el usuario decida salir.

```

Menú Principal
=====
MENÚ DE OPCIONES
=====
1. Consultar saldo
2. Retirar dinero
3. Depositar dinero
4. Ver historial de transacciones
5. Salir

Seleccione una opción: _

```

2.3. Validaciones del Menú Principal

- El sistema debe implementar validaciones específicas para asegurar un correcto funcionamiento:
- El menú principal debe mostrar únicamente las 5 opciones (1-5)
- No se deben aceptar letras, espacios en blanco ni caracteres especiales como entrada.
- Si el usuario ingresa un valor fuera de rango, debe mostrarse el mensaje:

```
Opción inválida, vuelva a intentarlo.
```

- Los montos de retiro o depósito deben ser valores numéricos positivos.

2.4. Estructura de datos para transacciones

- El sistema debe registrar las operaciones realizadas en un historial de transacciones.
- Cada transacción debe almacenarse en una estructura de datos que contenga al menos:
 - Tipo de operación (consulta, retiro, depósito)
 - Monto involucrado (si aplica)
 - Fecha y hora de la operación
- Este historial será utilizado en la opción 4. Ver historial de transacciones.

2.5. Opción 1: Consultar saldo

En el simulador, cada cliente inicia con un saldo predeterminado de ₡ 150.000,00, la primera vez que accede al sistema. Al seleccionar esta opción, se debe mostrar el saldo actual de forma clara y con el formato correcto, como se muestra a continuación:

```

-----
CONSULTAR SALDO

```

```
-----  
Su saldo actual es: ¢ 150.000,00
```

```
¿Desea volver al menú principal? Digite 1 para Si ó 0 para No
```

Nota importante: Si el usuario decide no continuar con la consulta, el sistema debe mostrar un mensaje y regresar automáticamente al Menú Principal:

```
Consulta cancelada. Regresando al Menú Principal
```

2.6. Opción 2: Retirar dinero

El sistema debe solicitar al usuario el monto a retirar (solo valor numérico) y mostrar un mensaje que indique si la transacción fue exitosa o rechazada. En el simulador cada cliente puede realizar un máximo de 5 transacciones, entre depósitos y retiros.

Ejemplo de pantalla con retiro exitoso:

```
-----  
                RETIRAR DINERO  
-----  
Ingrese monto a retirar: ¢ 20.000  
  
Procesando transacción...  
✓ Retiro exitoso.  
Saldo restante: ¢ 130.000,00  
  
¿Desea volver al menú principal? Digite 1 para Si ó 0 para No:  
  
En caso de fondos insuficientes:  
Procesando transacción...  
✗ Fondos insuficientes. Su saldo es ¢ 10.000,00  
  
¿Desea volver al menú principal? Digite 1 para Si ó 0 para No
```

En caso de fondos insuficientes

```
Procesando transacción ...  
Fondos insuficientes. Su saldo es de ¢ 10.000,00  
¿Desea volver al menú principal? Digite 1 para Si ó 0 para No
```

Notas importantes:

- Cada cliente puede realizar máximo 5 transacciones (retiros – depósitos), al llegar al límite, el sistema debe mostrar un mensaje y regresar al Menú Principal.

```
Ha alcanzado el límite de transacciones permitidas (5. Regresando al Menú Principal...
```

- Si el usuario digita 1, regresa al Menú Principal
- Si el usuario digita 0, el sistema debe mostrar un mensaje y finalizar la ejecución:

Gracias por usar el Simulador de Cajero Automático. Regrese pronto...

2.7. Opción 3: Depositar dinero

El sistema debe solicitar al usuario el monto a depositar (solo valor numérico) y mostrar mensaje que indique si la transacción fue exitosa. Cada cliente puede realizar un máximo de 5 transacciones en total, incluyendo depósitos y retiros.

```

-----
DEPOSITAR DINERO
-----
Ingrese monto a depositar: ₡ 50.000

Procesando transacción...
✓ Depósito exitoso.
Saldo actual: ₡ 200.000,00

¿Desea volver al menú principal? Digite 1 para Si ó 0 para No

```

Notas importantes:

- Cada cliente puede realizar máximo 5 transacciones (retiros – depósitos). Al llegar al límite, el sistema debe mostrar un mensaje y regresar automáticamente al Menú Principal.

Ha alcanzado el límite de transacciones permitidas (5. Regresando al Menú Principal...

- No se deben aceptar letras, caracteres especiales, montos en blanco o negativos. En caso de error:

Error: el monto ingresado no es válido. Intente nuevamente

- Si el usuario digita 1, regresa al Menú Principal
- Si el usuario digita 0, el sistema debe mostrar un mensaje y finalizar la ejecución:

Gracias por usar el Simulador de Cajero Automático. Regrese pronto...

2.8. Opción 4: Ver historial de transacciones

El sistema debe mostrar el detalle de las transacciones realizadas por el cliente en orden cronológico (de la más antigua a la más reciente). En el simulador cada cliente podrá tener un máximo 5 transacciones registradas.

```

-----
HISTORIAL DE TRANSACCIONES
-----
Fecha      | Tipo      | Monto      | Saldo después
-----+-----+-----+-----
01/08/2025 | Depósito | ₡ 50.000   | ₡ 200.000,00

```

```
02/08/2025 | Retiro | ¢ 20.000 | ¢ 150.000,00
03/08/2025 | Depósito | ¢ 10.000 | ¢ 170.000,00
```

```
¿Desea volver al menú principal? Digite 1 para Si ó 0 para No
```

Notas importantes

- Siempre se debe mostrar las transacciones de la más antigua a la más reciente.
- Si el cliente no ha realizado transacciones, el sistema debe mostrar el mensaje:

```
No existen transacciones registradas
```

- Si el usuario digita 1, regresa al Menú Principal
- Si el usuario digita 0, el sistema debe mostrar un mensaje y finalizar la ejecución:

```
Gracias por usar el Simulador de Cajero Automático. Regrese pronto...
```

2.9. Opción 5: Salir

El sistema debe presentar un mensaje cordial de despedida al cliente y cerra el programa.

El menú principal debe repetirse siempre después de cada operación, hasta que el usuario seleccione la opción 5 para salir.

La única forma de finalizar el programa debe ser mediante la selección #5 en el menú principal.

```
-----
GRACIAS POR USAR EL CAJERO
-----
¡Hasta pronto!

(Proceso finalizado)
```

- 2.10. Importante: Manejo de errores y mensajes: El sistema debe contemplar y mostrar pantallas o mensajes adecuados cuando ocurran errores o se seleccionen opciones que no estén definidas en el menú o en las funcionalidades implementadas. Estos mensajes deben seguir el mismo diseño y estilo visual de las pantallas ya presentadas en los puntos anteriores.

Ejemplos de situaciones a contemplar:

- Cuando el cliente no tiene transacciones registradas
- Cuando el cliente ingrese un PIN incorrecto
- Cuando se selecciona una opción inválida del menú, etc.

3. Requerimientos técnicos

3.1. Estructuras de control – Secuenciales. Repetición.

- 3.1.1. Bucle principal de menú. El sistema debe incluir un bucle principal que permita repetir el menú de opciones cuantas veces sea necesario, retornando siempre al mismo después de ejecutar cualquier opción, salvo cuando se seleccione la opción salir.

- 3.1.2. Iterar con los 4 PINs preestablecidos (variables para los máximos 5 clientes y/o Pines posibles). Ejemplo: `int PIN1 = 1234; int PIN2 = 2345; int PIN3 = 3456; int PIN4 = 4567;`
- 3.1.3. El sistema debe repetir la solicitud de información cuando se detecten valores incorrectos, nulos o en blanco, por ejemplo: PIN que no tenga exactamente 4 dígitos, montos vacíos, no numéricos o negativos, ingresos en blanco.
- 3.1.4. El usuario debe tener un máximo de 3 intentos para ingresar el PIN correctamente. Si no lo consigue, el sistema debe mostrar un mensaje y cerrar el programa automáticamente.
- 3.1.5. Debe implementarse la iteración sobre bucles para recorrer y mostrar el historial de hasta cinco transacciones. (retiros o depósitos). Ejemplo:

```
int tipoTrans1; double montoTrans1; double saldoDespues1;
...
int tipoTrans5; double montoTrans5; double saldoDespues5;
```

3.2. Estructuras de control – iterativas y de Decisión.

- 3.2.1. Selección de opción del menú. El sistema debe permitir al usuario seleccionar una de las 5 opciones del menú principal.
- 3.2.2. Validaciones de entrada. El sistema debe aplicar validaciones estrictas en cada entrada de datos por parte del usuario.
 - Comprobar que el PIN sea de 4 dígitos y numérico
 - Límite máximo de intentos de PIN, son 3 intentos.
 - Ningún campo puede estar vacío o sin ingresar.
 - Valores numéricos
 - Verificar que haya transacciones antes de mostrarlas en el historial.
 - Confirmar con el usuario antes de ejecutar operaciones importantes, por ejemplo, utilizando opciones como 'S'/'N' o 1/0 según lo indicado en los requerimientos funcionales)
 - Importante: El sistema debe mostrar mensajes claros al usuario cuando alguna validación no se cumple, indicando qué error cometió y solicitando la corrección.

4. Entregables

- 4.1. Debe entregar el Archivo `simuladorCajeroAutomatico.cpp` y/o proyecto completo en .zip. Código fuente con comentarios detallados de la lógica planteada.
- 4.2. (Opcional) Guía de uso en `README.md`, con comandos de compilación y ejemplos de ejecución.

Honestidad Académica



<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota Importante

- Cada estudiante es responsable del contenido que entrega.
- Si el archivo enviado no corresponde al proyecto correcto, no se aceptarán entregas posteriores fuera de la fecha establecida.
- Si se detecta que el contenido del archivo coincide con otro estudiante o no es de su autoría, se aplicarán las sanciones indicadas en el documento de Lineamientos ante casos de plagio disponible en la plataforma

Indicaciones Importantes

- Es obligatorio que incluya todo el directorio donde se encuentra Proyecto #1.
- El **Proyecto #1** debe estar desarrollado en **CodeBlocks** que es la herramienta oficial del curso.
- El programa debe ser modular, utilizando de la mejor manera funciones definidas por usted.
- Los trabajos deben realizarse en forma individual. Dentro del código del programa debe de indicar la documentación que explique cómo fue realizado el programa.
- Si utiliza código de algún ejemplo del libro, o de otra fuente que no sea de su autoría, debe de indicarlo.
- Comprima todos los archivos en un solo archivo .zip o .rar.
- **Nombre del archivo que envía:** debe ser nombre y primer apellido del estudiante, y nombre de la tarea. **Ejemplo: JuanRojas-proyecto1.**
- La entrega del **Proyecto #1 debe ser** en las fechas establecidas en la plataforma de aprendizaje en línea Moodle en el apartado que se indique.

Rúbrica de Evaluación

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significativos	No cumple o no presenta lo solicitado
Formato: Nitidez y presentación clara del código, incluyendo buena redacción y ortografía // Documentación interna adecuada dentro del código (comentarios explicativos)	5	3	1	0.1
Orden y claridad en el planteamiento (lógica estructurada). Cómo ordena las ideas para lograr la mejor solución, aplicando correctamente los conocimientos y herramientas vistos hasta el momento en el curso. Se evalúa correcta indentación del código, correcto uso de nombres significativos para las variables y estructura general de la lógica presentada.	10	5	3	0.1

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significativos	No cumple o no presenta lo solicitado
Estructuras de control – Secuenciales. Utiliza <i>if</i> , <i>if/else</i> y <i>switch</i> en la solución de forma adecuada. Según se solicita en los Requerimientos técnicos, punto #3 del enunciado. Considerando también consistencia con los Requerimientos funcionales, punto #2 del enunciado.	25	15	10	0.1
Estructuras de control - iterativas. Utiliza <i>while</i> , <i>do/while</i> y <i>for</i> en la solución de forma adecuada. Según se solicita en los Requerimientos técnicos, punto #3 del enunciado. Considerando también consistencia con los Requerimientos funcionales, punto #2 del enunciado.	25	15	10	0.1

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significativos	No cumple o no presenta lo solicitado
Validaciones. Implementa las validaciones necesarias y suficientes adicionales a la o las solicitadas en el enunciado. Según se solicita en los Requerimientos técnicos, punto #3 del enunciado. Considerando también consistencia con los Requerimientos funcionales, punto #2 del enunciado.	20	10	5	0.1
Impresión de información en pantalla (Calidad-validez datos/presentación tabulada). Presentación adecuada de entrada y salida de datos por pantalla. Solicitud de información, validaciones, presentación de lo mínimo solicitado. En específico en este punto también se evalúa toda la información que se indica en el enunciado, completa y correcta. Según se solicita en los Requerimientos funcionales, punto #2 del enunciado.	10	5	2	0.1

Interfaz de usuario en general. (NO GUI) - aplicación fácil usar e intuitiva (<i>mensajes</i>). Se refiere a evaluar la distribución general y uso de la pantalla, menús y dinámica de uso de la solución. Incluyendo lo mínimo según el ejemplo del enunciado o lo adicional que el estudiante entienda necesario.	5	3	1	0.1
TOTAL	100			